# ZetaERP — Manual Bug-Testing SOP

## Part 7 — Barcode (product barcodes, Code128/EAN encoding, label printing, POS scan)

| | |
|---|---|
| **Document** | QA-SOP-07 — Barcode |
| **Product** | ZetaERP / Zenith (multi-tenant POS / ERP) |
| **Environment under test** | Production — https://erp.zetaclantech.com |
| **Scope of this part** | The **Barcode** surface end-to-end: the dedicated **Barcode** module ( `/apps/zenith/barcode` — Dashboard, **Product Barcodes** grid, **Label Templates**, **Print Jobs**, the `ProductBarcode` multi-barcode model with EAN/UPC/Code128 types + checksum validation), the **POS barcode screens** ( `/apps/zenith/pos/Barcodes` bulk-assign grid + duplicate detection, `/apps/zenith/pos/BarcodeLabels` sticker printing), the **single-product code/barcode** field on EditProduct, the **Code128 / EAN encoders**, label rendering & printing, and **POS Terminal scan → cart** resolution. |
| **Audience** | Manual QA tester (no code knowledge assumed) |
| **Document owner** | Platform / Solo-dev |
| **Status** | Active |
| **Last updated** | 2026-07-01 |

### CONTENTS


1. Purpose
2. Critical rules for the tester (read before starting)
3. Test environment & preconditions
4. Test data
5. Conventions & severity
6. Flow map (barcode lifecycle)
7. BARCODE DASHBOARD
8. PRODUCT BARCODES (rich ProductBarcode model)
9. LABEL TEMPLATES & PRINT JOBS
   - 9.1 Label Templates
   - 9.2 Print Jobs
10. POS BARCODES GRID & LABEL PRINTING (the pseudoId path)
    - 10.1 POS Bulk Barcodes grid
    - 10.2 POS Barcode Labels (Code128 sticker printing)
11. THE TWO-SYSTEM GAP & SINGLE-PRODUCT EDIT
    - 11.1 Single-product code edit
    - 11.2 The integration gap (verify explicitly)
12. POS TERMINAL SCAN → CART (resolution)
13. MULTI-TENANT ISOLATION
14. UI / ACCESSIBILITY / SECURITY & defect template
15. Areas to probe hardest (attention map)
16. Teardown (mandatory)

## 1. Purpose

This Standard Operating Procedure tells a tester **exactly how to manually test the Barcode surface A→Z**: creating/assigning barcodes (single, bulk, auto-generated), validating **EAN-13/EAN-8/UPC-A checksums**, enforcing **uniqueness** and **primary** selection, designing **label templates**, **previewing + printing** scannable labels, and — most importantly — **scanning a barcode at the POS Terminal and getting the right product into the cart**. The barcode is what makes fast checkout work, so the highest-value tests are the round-trips: *assign a barcode → print a label → scan that label → correct product appears*.

> **Read this first (the single most important structural fact):** there are **two barcode systems** that do **not** fully talk to each other.
>
> - The **Barcode module** ( `/apps/zenith/barcode` ) stores rich barcodes in the `ProductBarcode` entity (multiple per product, typed EAN/UPC/Code128, checksum-validated).
> - **POS billing** (the Terminal scan, the `pos/Barcodes` grid, `pos/BarcodeLabels` ) uses the product's `pseudoId` ("Item Code / Barcode") field.
>
> **POS scanning resolves against `pseudoId` only — it does not query `ProductBarcode` .** So a barcode you add in *Product Barcodes* (FindBarcode) will **not** be found at the till unless the same value is also the product's `pseudoId` . Verifying this gap (§11, BC-060) is a primary objective of this SOP.

## 2. Critical rules for the tester (read before starting)

> ⚠️ **This is a LIVE commercial system. Barcodes drive real checkout.**

1. **NEVER edit, reassign, or delete a real product's barcode/ `pseudoId` on the live flagship tenant **MARUTHI (org `100000` )** or any live tenant. A changed `pseudoId` means that product **stops scanning at the till**. Test on a **throwaway tenant** with throwaway products.
2. **Do not print to a real shared printer** during tests — use Print-to-PDF / a preview, or the browser print dialog's cancel. Don't spool test sheets to a store printer.
3. **Duplicate/uniqueness tests must be isolated** — a duplicate `pseudoId` on a live tenant can cause the wrong product to be billed. Only create duplicates on throwaway products, and clean them up.
4. **Clean up every test barcode, label template, print job, and product** you create (§16), or hand the list to the dev.
5. **Barcode data is per-tenant** — verify isolation (§12); a cross-tenant barcode/product resolution is **Critical**.
6. Record the **exact scanned/typed string** (including leading zeros and any scanner suffix like Enter) for every scan case — leading zeros and hidden characters are where barcode bugs hide.

## 3. Test environment & preconditions

Screens (open from the app menu):

| Area | Screen (URL) | App | Purpose |
|---|---|---|---|
| Barcode Dashboard | `/apps/zenith/barcode/BarcodeDashboard` | /apps (Bootstrap) | Overview + quick lookup |
| **Product Barcodes** | `/apps/zenith/barcode/FindBarcode` | /apps | Add / auto-generate / manage `ProductBarcode` (typed, multi) |
| Label Templates | `/apps/zenith/barcode/LabelTemplates` | /apps | CRUD label layouts (size, columns, show name/price/SKU/MRP) |
| Print Jobs | `/apps/zenith/barcode/PrintJobs` | /apps | Print-job history + render labels |
| **POS Bulk Barcodes** | `/apps/zenith/pos/Barcodes` | /apps | Assign `pseudoId` per product + duplicate detection |
| **POS Barcode Labels** | `/apps/zenith/pos/BarcodeLabels` | /apps | Print Code128 stickers (SVG) by product |
| Single-product code | EditProduct → "Item Code / Barcode" ( `pseudoId` ) | **/qapps (Quasar)** | Edit one product's scan code |
| **POS Terminal** | `/apps/zenith/pos/Terminal` | /apps | Scan → add to cart ( `lookupProducts` , `pseudoId` ) |

**Storage:** `Product.pseudoId` (single scan code, per `ownerPartyId`) + `zenith.erp.barcode.ProductBarcode` (many per product; unique on `organizationPartyId + barcodeValue`; types `BctEan13/BctEan8/BctUpcA/BctUpcE/BctCode128/BctCode39/BctQrCode/BctInternal`).

**Encoders:** a self-contained **Code128-B SVG** encoder in `pos-barcode-labels.js` (start 104, checksum `sum%103`, stop 106) and the **JsBarcode** library (`renderLabels` in the barcode module). EAN/UPC **checksum** validation lives in `add#ProductBarcode`.

**Set up on the throwaway tenant:** ~5 products, some with `pseudoId` set and some without; a real handheld **USB/Bluetooth barcode scanner** if available (a scanner just types the digits + Enter — you can simulate by typing). Have the **POS Terminal**, **Product Barcodes**, **POS Barcodes** and **Barcode Labels** screens reachable.

**Browsers:** full pass on Desktop Chrome; spot-check Edge/Safari/mobile. **Tools:** a barcode scanner (or keyboard), a phone barcode-scanner app to read a printed/preview label, DevTools (Console/Network), and Print-to-PDF.

---

## 4. Test data

| Label | Use |
|---|---|
| Throwaway tenant | Non-MARUTHI org for all barcode testing |
| `PROD_A` … `PROD_E` | Throwaway products (some with `pseudoId`, some blank) |
| Valid EAN-13 | e.g. `5901234123457` (valid checksum) |
| Bad-checksum EAN-13 | e.g. `5901234123450` (last digit wrong) |
| Valid UPC-A | e.g. `036000291452` |
| Code128 value | e.g. `ZEN-ABC-001` (alphanumeric, variable length) |
| Leading-zero EAN | e.g. `0123456789012` |
| Duplicate value | the same code deliberately put on two throwaway products |
| Auto-generated | `generate#InternalBarcode` → `ZEN` + 9 digits (Code128/internal) |

**EAN-13 checksum (to verify by hand):** from the right, weight the 12 data digits alternately ×1,×3,… ; check digit = `(10 − (sum mod 10)) mod 10`. (The service uses the mod-10 alternating 3/1 rule.)

---

## 5. Conventions & severity

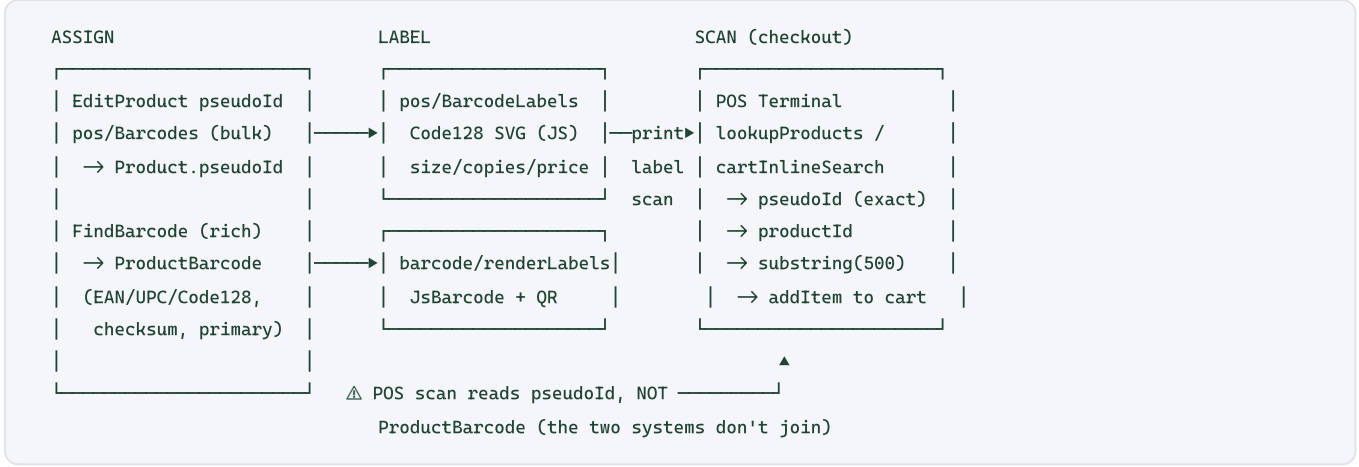
**Case Type:** `FUNC` · `ENC` (encoder/checksum correctness) · `SCAN` (POS resolution) · `EDGE` · `UI` · `A11Y` · `SEC` · `ISO` · `PERF`.

| Sev | Meaning | Example |
|---|---|---|
| **S1 — Critical** | Wrong product billed from a scan; cross-tenant barcode/product access; a printed label that doesn't scan or scans as the wrong code | duplicate `pseudoId` silently bills the wrong item; a Code128 label decodes to a different string |
| **S2 — High** | A barcode flow broken or validation missing | EAN checksum not enforced; `add#ProductBarcode` lets a duplicate through; scan of a valid `pseudoId` returns no match |
| **S3 — Medium** | Wrong/loose behaviour, limited impact | primary flag not exclusive; label template field ignored; no-match error unclear |
| **S4 — Low** | Cosmetic / minor UX | label spacing; grid column misaligned; dashboard count off |

**Logging a defect** (template §14): Case ID, steps, **exact code string** (with leading zeros), product(s) involved, tenant, browser, screenshot (incl. the rendered/printed label and a phone-scan of it where relevant), Console/Network. Evidence is **mandatory** for ENC, SCAN, ISO and SEC.

---

### 6. Flow map (barcode lifecycle)




# 7. BARCODE DASHBOARD

**Screen:** `/apps/zenith/barcode/BarcodeDashboard` .

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-001 | FUNC | Dashboard renders | Open the dashboard | Counts (active barcodes, templates, pending jobs) render; org-scoped; no console errors | S3 |
| BC-002 | FUNC | Quick lookup | Enter a known barcode in the quick-lookup form | Resolves to the right product; unknown → clear "not found" | S2 |
| BC-003 | UI | Empty state | New tenant, no barcodes | Zero counts, no NaN/crash | S3 |

# 8. PRODUCT BARCODES (rich `ProductBarcode` model)

**Screen:** `/apps/zenith/barcode/FindBarcode` . Services `add#ProductBarcode` , `generate#InternalBarcode` , `get#ProductBarcodes` , `lookup#ProductByBarcode` .

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-010 | FUNC | List barcodes | Open FindBarcode | Grid of product / SKU / value / type / primary; org-scoped | S3 |
| BC-011 | FUNC | Add a Code128 barcode | Add a `BctCode128` value to `PROD_A` | Created, linked to product | S2 |
| BC-012 | ENC | **EAN-13 checksum accepted** | Add a **valid** EAN-13 ( `5901234123457` ) | Accepted | S2 |
| BC-013 | ENC | **EAN-13 checksum rejected** | Add a **bad-checksum** EAN-13 ( `5901234123450` ) | **Rejected** with a clear checksum error | S1 |
| BC-014 | ENC | EAN-8 / UPC-A checksum | Add valid + invalid EAN-8 and UPC-A | Valid accepted, invalid rejected (12/8-digit + checksum) | S2 |
| BC-015 | ENC | Wrong length / non-numeric EAN | 12-digit "EAN-13", or letters in an EAN | Rejected (regex `[0-9]{13}` etc.) | S2 |
| BC-016 | FUNC | **Uniqueness within tenant** | Add a value already used by another product (same org) | **Rejected** ("barcode already used") — unique on `organizationPartyId+barcodeValue` | S1 |
| BC-017 | FUNC | Multiple barcodes / product | Add UPC + EAN + internal to `PROD_A` | All stored; product has several barcodes | S3 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-018 | FUNC | **Primary is exclusive** | Mark a 2nd barcode `isPrimary=Y` | The previous primary flips to N (only one primary per product) | S2 |
| BC-019 | FUNC | Auto-generate internal | `generate#InternalBarcode` for `PROD_B` | `ZEN` +9-digit Code128 value, `isActive=Y`, not primary | S3 |
| BC-020 | FUNC | `lookup#ProductByBarcode` | Look up an added value | Returns productId, name, price (mrpAmount), type; unknown → `found=false` | S2 |
| BC-021 | EDGE | Deactivate barcode | Set a barcode `isActive=N` | Excluded from active lookups | S3 |

# 9. LABEL TEMPLATES & PRINT JOBS

## 9.1 Label Templates

**Screen:** `/apps/zenith/barcode/LabelTemplates`. Entity `BarcodeLabel` (width/height mm, `columnsPerRow`, `showProductName/showPrice/showSku/showMrp`, default type, `isDefault`).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-030 | FUNC | Create template | New template (50×25, 3 cols, show name+price) → Save | Persists; appears in the list | S3 |
| BC-031 | FUNC | Default template | Mark one `isDefault` | Only one default; used as the pre-selected layout | S3 |
| BC-032 | UI | Show/hide fields | Toggle name/price/SKU/MRP | The rendered label reflects each toggle (note: `showMrp` may be unused — verify) | S3 |
| BC-033 | EDGE | Silly dimensions | 0×0 or huge mm, 0 columns | Handled sensibly (no divide-by-zero / blank render) | S3 |

## 9.2 Print Jobs

**Screen:** `/apps/zenith/barcode/PrintJobs`. Services `create#PrintJob` (+ `renderLabels` transition using JsBarcode + QR).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-034 | FUNC | Create print job | Pick a template + products×qty → create | `BarcodePrintJob` (Pending), one item per product with its **primary** barcode (else productId) | S2 |
| BC-035 | FUNC | Render labels | Open renderLabels for a job | HTML page renders each barcode via JsBarcode; QR renders where type=QR | S2 |
| BC-036 | ENC | Type→format mapping | Job with EAN13/Code128/QR items | Each renders in the correct symbology (BctEan13→EAN13, BctInternal→CODE128, etc.) | S2 |
| BC-037 | FUNC | Mark printed | Mark job Completed | Status updates; history correct | S3 |
| BC-038 | EDGE | Product with no primary barcode | Job includes a product with no barcode | Falls back to productId (or is flagged) — no crash | S3 |

# 10. POS BARCODES GRID & LABEL PRINTING (the `pseudoId` path)

## 10.1 POS Bulk Barcodes grid

**Screen:** `/apps/zenith/pos/Barcodes` (writes `Product.pseudoId`; `saveBarcode` transition; JS Enter-to-save; duplicate scan).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-040 | FUNC | Assign a code | Type a code for `PROD_A`, press Enter | Saved to `pseudoId`; row shows "OK"; AJAX success | S2 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-041 | FUNC | **Duplicate detection** | Assign the **same** code to `PROD_A` and `PROD_B` | Grid flags both as **"Duplicate"** with a count/warning | S1 |
| BC-042 | FUNC | Duplicate blocked on save? | Try to save a code already used by another product | Record actual — does `saveBarcode` **reject** it, or just warn? (a saved duplicate = wrong-product-at-till risk) | S1 |
| BC-043 | FUNC | Clear a code | Blank a product's code → save | `pseudoId` cleared; row shows "No code" | S3 |
| BC-044 | UI | Search / filter / paginate | Filter by product/category, page through | Correct filtered set; codes persist across pages | S3 |
| BC-045 | ISO | Grid org-scoped | List the grid as tenant A | Only A's products; duplicate scan spans only A | S1 |

### 10.2 POS Barcode Labels (Code128 sticker printing)

**Screen:** `/apps/zenith/pos/BarcodeLabels` ( `pos-barcode-labels.js` Code128 SVG; sizes 50×25 / 38×25 / 65×35 / 100×50 mm; copies; show price; preview + iframe print).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-050 | FUNC | Only coded products selectable | Open the screen | Products **without** `pseudoId` show ❌ / can't be ticked; coded ones tickable | S2 |
| BC-051 | FUNC | Preview labels | Tick 2 products, size 50×25, copies 2, show price → Preview | 4 labels render in the sheet; name + barcode + code text + ₹price | S2 |
| BC-052 | ENC | **Printed label scans back correctly** | Preview/print a label, scan it with a phone app **or** the handheld into a text box | The decoded string **exactly equals** the product's `pseudoId` (incl. leading zeros) | S1 |
| BC-053 | ENC | Code128 checksum correctness | Compare the SVG encoder output with JsBarcode for the same value | Byte-identical bars (encoder proven equal to JsBarcode) — decodes identically | S1 |
| BC-054 | FUNC | Label sizes & module width | Try 38×25 and 100×50 | Barcode scales (module width by label width); still scannable | S3 |
| BC-055 | FUNC | Copies & batch | 3 products × 3 copies | 9 labels, grid wraps, `page-break-inside:avoid` (no split labels) | S3 |
| BC-056 | UI | Print pipeline | Click Print | Hidden iframe → browser print dialog; **no popup blocked**; cancel-safe | S3 |
| BC-057 | EDGE | Long / special-char value | A long Code128 value; a value with `& < ' \` | SVG width scales; value HTML/JS-escaped (no broken render / injection) | S2 |

---

# 11. THE TWO-SYSTEM GAP & SINGLE-PRODUCT EDIT

### 11.1 Single-product code edit

**Screen:** EditProduct → "Item Code / Barcode" = `pseudoId` (**/qapps Quasar**).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-058 | FUNC | Edit pseudoId | Set a product's Item Code and Save | `pseudoId` persists; field editable (Quasar input) | S2 |
| BC-059 | EDGE | No validation on pseudoId | Enter spaces / very long / duplicate here | Record actual — this field has **no** checksum/uniqueness guard (unlike `ProductBarcode` ) | S3 |

### 11.2 The integration gap (verify explicitly)

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-060 | SCAN | **ProductBarcode value does NOT scan at POS** | Add a value **only** in *Product Barcodes* ( `ProductBarcode` ), leave `pseudoId` blank → go to the Terminal and scan it | ⚠ Expected (current design): **no match** — POS reads `pseudoId` , not `ProductBarcode` . Confirm, and flag as the system gap; the two must be reconciled before relying on the rich barcode module for billing | S1 |
| BC-061 | SCAN | Same value as pseudoId **does** scan | Set that value as the product's `pseudoId` too → scan | Product resolves and adds to cart | S1 |

---

# 12. POS TERMINAL SCAN → CART (resolution)

**Screen:** `/apps/zenith/pos/Terminal` ( `lookupProducts` , `cartInlineSearch` ; resolves `pseudoId` exact → `productId` → substring on ≤500).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| BC-070 | SCAN | Exact `pseudoId` scan | Scan a product's exact code into the cart search | Correct product added to cart immediately | S1 |
| BC-071 | SCAN | Scanner Enter key | Scan with a real handheld (types digits + Enter) | Enter triggers add; no stray newline in the field | S2 |
| BC-072 | SCAN | Type-ahead by name/code | Type a partial name / code | `lookupProducts` returns top matches (pseudoId/name/productId substring) | S3 |
| BC-073 | SCAN | **Unknown barcode** | Scan a code with no product | Clear "not found" message; **nothing** added; cart unchanged | S2 |
| BC-074 | SCAN | **Duplicate `pseudoId`** | Two products share a `pseudoId` ; scan it | ⚠ `.one()` returns the **first** match → **wrong product risk**. Record which product is billed; this is why BC-041/042 matter | S1 |
| BC-075 | SCAN | Leading zeros | Product `pseudoId` `0123456789012` ; scan it | Resolves exactly (leading zeros preserved end-to-end) | S2 |
| BC-076 | PERF | Large catalogue | Tenant with ~3000 products; scan | Resolves fast (indexed pseudoId path, not a full scan every time) | S3 |
| BC-077 | EDGE | Whitespace / case | Scan with trailing space or different case | Record actual — does trim/case-insensitivity help or cause a false match? | S3 |

---

# 13. MULTI-TENANT ISOLATION

`ProductBarcode` , `BarcodeLabel` , `BarcodePrintJob` , `BarcodePrintJobItem` are org-filtered ( `MANTLE_ACTIVE_ORG` set); `Product` by `ownerPartyId` .

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| ISO-001 | ISO | Barcodes scoped | Tenant A lists Product Barcodes / labels / jobs | Only A's rows; none of B's | S1 |
| ISO-002 | ISO | Same value, two tenants | A and B each have a product with value `123456` | Each resolves to **its own** product; no cross-over | S1 |
| ISO-003 | ISO | POS scan scoped | A's Terminal scans a code that only B's product has | **No match** on A (lookup filters by A's org) | S1 |
| ISO-004 | ISO | Direct-URL / lookup | A calls `lookup#ProductByBarcode` for B's value | Not found / not B's product | S1 |
| ISO-005 | ISO | Print-job render scoped | A opens renderLabels for a job id belonging to B | Blocked / not found | S1 |

---

# 14. UI / ACCESSIBILITY / SECURITY & defect template

| ID | Type | Title | Check | Expected result | Sev |
|---|---|---|---|---|---|
| BC-090 | UI | Desktop layout | Grids + label sheet at 1920/1366 | Aligned; label preview grid correct | S4 |

| ID | Type | Title | Check | Expected result | Sev |
|---|---|---|---|---|---|
| BC-091 | UI | Mobile layout | POS Barcodes / Labels at 360–390px | Usable; no horizontal overflow | S3 |
| BC-092 | A11Y | Keyboard | Tab through the grid code inputs; Enter saves | Logical order; Enter-to-save works; visible focus | S3 |
| BC-093 | SEC | XSS via product name/value | Product name / barcode value `<img src=x onerror=alert(1)>` | Escaped in grids, labels (SVG/HTML) and print output; never executes (BC-057) | S1 |
| BC-094 | SEC | Save-barcode authz/CSRF | (Dev-assisted) POST `saveBarcode` / `add#ProductBarcode` for another tenant's product or without a token | Rejected (org-checked + session token) | S1 |

```
Defect ID:        ZE-BC-____
Linked Case ID:   BC-0__ / ENC / SCAN / ISO-0__
Title / Severity / Type:
Environment:      Prod | <browser + scanner model> | tenant: <org>
Exact code string: "<value, show leading zeros / suffix>"   Type: EAN13/Code128/…
Product(s):       <productId(s) + pseudoId(s)>
Expected / Actual: (attach screenshot of grid + rendered label + a phone-scan of the label)
Console / Network: (attach)
```

## 15. Areas to probe hardest (attention map)

1. **The two-system gap (BC-060/061)** — a `ProductBarcode` value is **not** found by the POS scan (which uses `pseudoId`). This is the single most important structural issue: the rich barcode module and the billing scan don't share a source of truth. Prove it and flag it for reconciliation.
2. **Duplicate `pseudoId` → wrong product billed (BC-041/042/074)** — there is **no DB uniqueness on `pseudoId`** (only `ProductBarcode` has the unique index). The bulk grid *warns* but confirm whether it *prevents* a duplicate save; `.one()` at the till silently picks the first match.
3. **EAN/UPC checksum enforcement (BC-013/014/015)** — a bad-checksum EAN must be rejected by `add#ProductBarcode`; the plain `pseudoId` field has **no** such guard (BC-059).
4. **Printed-label-scans-back correctness (BC-052/053)** — the whole point: the Code128 SVG must decode to exactly the stored code. Scan a real printed/preview label with a phone; verify byte-for-byte, including leading zeros.
5. **POS resolution order & no-match (BC-070/073/074/075)** — exact `pseudoId` first, then productId, then substring; unknown must add nothing; leading zeros preserved.
6. **Multi-tenant isolation (§13)** — same barcode value in two tenants must resolve to each tenant's own product; A's till must never find B's product.
7. **Escaping in labels/print (BC-057/093)** — product names/values flow into SVG + inline JS + print HTML; verify escaping (XSS + broken render).

## 16. Teardown (mandatory)

1. **List everything created** — `ProductBarcode` rows, `pseudoId` values you set, label templates, print jobs, throwaway products — with ids.
2. **Delete/clear** them (or hand the dev the list). In particular, **clear any test `pseudoId`** and **remove duplicate codes** so nothing is left that could mis-scan.
3. Delete any **preview/print PDFs** generated locally; don't leave test sheets on a printer queue.
4. Confirm you **never touched MARUTHI** (org `100000`) or any live product's barcode/ `pseudoId`.
5. Attach the run sheet(s) + defect ids to this part's report.

*End of Part 7 — Barcode. Related: Part 3 — POS (the Terminal scan + checkout); Part 5 — Inventory (EditProduct holds the `pseudoId`). The headline items to raise with the dev: reconcile the `ProductBarcode` module with the POS `pseudoId` scan (one source of truth), and decide whether `pseudoId` should get the same uniqueness + checksum guards the `ProductBarcode` model already enforces.*